

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΠΛΗΡΟΦΟΡΙΚΗΣ**



Τομέας: Υπολογιστική Υψηλών Επιδόσεων
Εργαστήριο: Εργαστήριο Πληροφοριακών Συστημάτων
Υψηλών Επιδόσεων

**Συγκριτική Ανάλυση και Πειραματική
Αξιολόγηση Συστημάτων Παράλληλου
Προγραμματισμού Βασισμένου σε
Εργασίες**

Διπλωματική Εργασία

του φοιτητή

Γεωργίου Τσάβου του Κωνσταντίνου

Αριθμός Μητρώου: 1084606

Επιβλέπων: Παναγιώτης Χατζηδούκας, Αναπληρωτής Καθηγητής,
Πανεπιστήμιο Πατρών

Πάτρα, Ιούνιος 2026

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα:

**Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού
Βασισμένου σε Εργασίες**

του φοιτητή του Τμήματος Μηχανικών Ηλεκτρονικών Υπολογιστών και
Πληροφορικής

Γεωργίου Τσάβου του Κωνσταντίνου

(Α.Μ.: 1084606)

παρουσιάστηκε δημόσια και εξετάστηκε επιτυχώς από την
τριμελή εξεταστική επιτροπή στις __/__/____.

Ο Επιβλέπων

Μέλος της Επιτροπής

Μέλος της Επιτροπής

**Παναγιώτης
Χατζηδούκας**
Αναπληρωτής Καθηγητής

**Σπυρίδων
Σιούτας**
Καθηγητής

**Γεώργιος
Παπαδημητρίου**
Επίκουρος Καθηγητής

Στοιχεία Διπλωματικής Εργασίας

Θέμα:

**Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού
Βασισμένου σε Εργασίες**

Φοιτητής:

Γεώργιος Τσάβος του Κωνσταντίνου

Ομάδα Επίβλεψης:

Παναγιώτης Χατζηδούκας

Αναπληρωτής Καθηγητής, Πανεπιστήμιο Πατρών

Εργαστήριο:

Εργαστήριο Πληροφοριακών Συστημάτων Υψηλών Επιδόσεων

Περίληψη

Λέξεις-κλειδιά: HPC, Διεργασίες, task-parallelization, heterogeneity

Abstract

Keywords: HPC, Διεργασίες, task-parallelization, heterogeneity

Ευχαριστίες

Περιεχόμενα

Κατάλογος σχημάτων	viii
Κατάλογος πινάκων	ix
1 Εισαγωγή	1
1.1 Σκοπός και Αντικείμενο της Εργασίας	1
1.2 Δομή της Εργασίας	2
2 Θεωρητικό Υπόβαθρο	5
Βιβλιογραφία	7

Κατάλογος σχημάτων

Κατάλογος πινάκων

Κεφάλαιο 1

Εισαγωγή

1.1 Σκοπός και Αντικείμενο της Εργασίας

Η ραγδαία εξέλιξη της υπολογιστικής επιστήμης και η ανάγκη για την ανάλυση όλο και μεγαλύτερου όγκου δεδομένων έχουν καταστήσει τον παράλληλο προγραμματισμό αναπόσπαστο κομμάτι της σύγχρονης έρευνας και τεχνολογίας. Καθώς η φυσική αύξηση της συχνότητας των επεξεργαστών (ρολόι) έχει φτάσει στα όριά της, η σύγχρονη υπολογιστική ισχύς πηγάζει πλέον κυρίως από αρχιτεκτονικές πολλαπλών πυρήνων (multi-core) και συστήματα συμμετρικής ή μη-συμμετρικής πολυεπεξεργασίας. Παράλληλα, γλώσσες υψηλού επιπέδου όπως η Python κυριαρχούν στους επιστημονικούς υπολογισμούς λόγω της ευκολίας χρήσης τους και του πλούσιου οικοσυστήματος βιβλιοθηκών τους, όμως η αποτελεσματική εκμετάλλευση του σύγχρονου υλικού (hardware) μέσω αυτής αποτελεί μια μείζονα πρόκληση, κυρίως λόγω εγγενών περιορισμών της γλώσσας στον ταυτόχρονο προγραμματισμό.

Αντικείμενο της παρούσας διπλωματικής εργασίας είναι η συγκριτική ανάλυση και πειραματική αξιολόγηση δύο διαφορετικών συστημάτων παράλληλου προγραμματισμού βασισμένου σε εργασίες (task-based parallel programming). Το μοντέλο αυτό προσφέρει ένα υψηλό επίπεδο αφαίρεσης, όπου το πρόγραμμα διασπάται σε ανεξάρτητες μονάδες εκτέλεσης, τις "εργασίες" (tasks). Ο προγραμματιστής ορίζει τις εργασίες και τις μεταξύ τους εξαρτήσεις, ενώ το σύστημα χρόνου εκτέλεσης (runtime system) αναλαμβάνει δυναμικά να τις δρομολογήσει στους διαθέσιμους πυρήνες, να διαχειριστεί τη μνήμη και να εξισορροπήσει τον φόρτο εργασίας, απαλλάσσοντας τον χρήστη από τον χειροκίνητο και επίπονο έλεγχο των νημάτων.

Σκοπός της εργασίας είναι να μελετηθεί πώς τα συστήματα αυτά ανταποκρίνονται στις σύγχρονες προκλήσεις του υλικού και του λογισμικού. Η πρώτη μεγάλη πρόκληση αφορά τις αρχιτεκτονικές **NUMA (Non-Uniform Memory Access)**, οι οποίες κυριαρχούν στα συστήματα υψηλών επιδόσεων (HPC). Σε αυτές, η μνήμη είναι χωρισμένη σε τοπικά τμήματα ανά επεξεργαστή, με αποτέλεσμα η πρόσβαση σε απομακρυσμένα τμήματα μνήμης να προκαλεί καθυστερήσεις και συμφόρηση. Τα task-based συστήματα πρέπει, επομένως, να έχουν επίγνωση της τοπολογίας (NUMA-awareness) για να διατηρούν

την τοπικότητα των δεδομένων. Η δεύτερη και σημαντικότερη πρόκληση στο περιβάλλον της Python είναι το **Global Interpreter Lock (GIL)**, ένας μηχανισμός που επιτρέπει την εκτέλεση κώδικα Python από ένα μόνο νήμα ανά πάσα στιγμή. Σε *cpu-bound* εφαρμογές, η χρήση απλών νημάτων οδηγεί σε *GIL thrashing* (διαρκείς εναλλαγές για την απόκτηση του lock), προκαλώντας κατακρήμνιση της χρήσης της CPU (ακόμα και στο 1.5%) και ακυρώνοντας τον παραλληλισμό.

Για την αντιμετώπιση αυτών των προβλημάτων, η εργασία εξετάζει και συγκρίνει δύο πλατφόρμες με θεμελιωδώς διαφορετικές αρχιτεκτονικές φιλοσοφίες:

- **StarPU (και η διεπαφή StarPUPY):** Ένα σύστημα χαμηλού επιπέδου σε C, σχεδιασμένο για *shared memory* και ετερογενή συστήματα. Θα επικεντρωθούμε στην Python διεπαφή του, StarPUPY για την πειραματική αξιολόγηση.
- **torcpy:** Ένα Python-native σύστημα βασισμένο στο κατανεμημένο πρότυπο MPI (μέσω της `mpi4py`), ιδανικό για κατενεμημένα συστήματα.

Η πειραματική αξιολόγηση και η συνεισφορά της εργασίας εστιάζουν στην υλοποίηση και σύγκριση των δύο συστημάτων πάνω σε τρεις αντιπροσωπευτικές *cpu-bound* εφαρμογές: έναν αλγόριθμο βελτιστοποίησης (CMA-ES με επίλυση Poisson PDE), μια ροή επεξεργασίας εικόνας (Gabor Filters), και μια προσομοίωση Monte Carlo (Basket Options). Μέσα από αυτές τις εφαρμογές, η εργασία ποσοτικοποιεί τον χρόνο εκτέλεσης σε συνθήκες ισχυρού παραλληλισμού (64 πυρήνες), αναδεικνύοντας τα όρια και τις βέλτιστες πρακτικές χρήσης του κάθε συστήματος.

1.2 Δομή της Εργασίας

Η παρούσα διπλωματική εργασία είναι οργανωμένη σε επτά συνολικά κεφάλαια, ακολουθώντας μια λογική πορεία από τη θεωρητική θεμελίωση στην πρακτική εφαρμογή και την ανάλυση:

- **Κεφάλαιο 2 -- Θεωρητικό Υπόβαθρο:** Παρουσιάζεται το θεωρητικό υπόβαθρο γύρω από τα *task runtimes*, την ετερογένεια και τις προκλήσεις των αρχιτεκτονικών NUMA.
- **Κεφάλαιο 3 -- Το Σύστημα StarPU:** Αναλύονται οι δυνατότητες και η αρχιτεκτονική του StarPU σε περιβάλλον C, καθώς και ο τρόπος διαχείρισης των πόρων.
- **Κεφάλαιο 4 -- Η Διεπαφή StarPUPY:** Εξετάζεται η μεταφορά του StarPU στην Python, οι μέθοδοι διαχείρισης του GIL και οι ιδιαιτερότητες του χρονοπρογραμματισμού του.

- **Κεφάλαιο 5 -- Το Σύστημα torcpy:** Αναλύεται το μοντέλο εκτέλεσης του torcpy, το API του, ο τρόπος δρομολόγησης των tasks και η χρήση του MPI για την απομόνωση διεργασιών.
- **Κεφάλαιο 6 -- Εφαρμογές και Μεθοδολογία:** Περιγράφεται αναλυτικά ο κώδικας και η μαθηματική δομή των τριών εφαρμογών (CMA-ES, Image Processing, Monte Carlo), καθώς και ο τρόπος λήψης των μετρήσεων.
- **Κεφάλαιο 7 -- Πειραματική Αξιολόγηση και Συμπεράσματα:** Παρουσιάζονται τα διαγράμματα και τα αποτελέσματα των μετρήσεων, αναλύονται τα φαινόμενα saturation και κλιμάκωσης, και συνοψίζονται τα τελικά συμπεράσματα της σύγκρισης.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

Βιβλιογραφία

- [1] Panagiotis E Hadjidakas κ.ά. ``torcpy: Supporting task parallelism in Python". Στο: SoftwareX 12 (2020), σ. 100517.

